

Sapientia Erdélyi Magyar Tudományegyetem  
Marosvásárhelyi Kar  
Számítástechnika szak

# Bus4U - Szerver oldal

## DIPLOMADOLGOZAT

**Témavezető:**  
Dr. Szántó Zoltán

**Hallgató:**  
Zsigmond Attila

2024

Universitatea „Sapientia” din Cluj-Napoca  
Facultatea de Științe Tehnice și Umaniste din Târgu-Mureș  
Specializarea Calculatoare

# Bus4U - Partea de Server

## PROIECT DE DIPLOMĂ

**Coordonator științific:**  
Dr. Szántó Zoltán

**Student:**  
Zsigmond Attila

2024

## Kivonat

Scopul aplicației web este de a face sistemele moderne de transport public mai prietenoase cu utilizatorul și mai eficiente, cu un accent deosebit pe planificarea călătoriilor, achiziționarea de bilete și urmărirea autobuzelor. Aplicația permite utilizatorilor să își planifice călătoriile în avans în cadrul rețelei de transport public dintr-un oraș sau o regiune specifică. Utilizatorii își pot achiziționa biletele pentru o rută selectată și pot vizualiza orele de plecare și sosire ale curselor, defalcat pe orașe și stații, asigurând astfel flexibilitate și informații de călătorie precise.

Aplicația oferă, de asemenea, o hartă interactivă care permite urmărirea în timp real a pozițiilor curente ale autobuzelor. Această funcție este deosebit de utilă atunci când apar schimbări neprevăzute în program, deoarece utilizatorii pot fi informați la timp despre întârzieri sau modificări ale traseelor. Pe lângă urmărirea în timp real, sistemul oferă locațiile geografice precise ale stațiilor de autobuz individuale, pe care utilizatorii le pot filtra în funcție de orașe sau rute, facilitând astfel planificarea la care stație să ajungă.

O aplicație separată care rulează pe terminalele POS aflate pe autobuze se ocupă de validarea biletelor și de partajarea continuă a pozițiilor autobuzelor cu serverul central. Această funcție asigură că toți participanții la sistem au informații actualizate, permițând utilizatorilor să rămână informați despre locațiile în timp real ale autobuzelor.

O altă componentă importantă a aplicației este interfața de administrare utilizată de companiile de autobuze. Această interfață permite gestionarea programelor, adăugarea de noi rute și stații și actualizarea datelor existente. În plus, informațiile despre angajați, inclusiv detaliile șoferilor și ale altui personal, precum și datele tehnice și de operare ale autobuzelor, pot fi gestionate aici. Acest sistem de administrare asigură funcționarea eficientă și întreținerea programelor, permițând companiilor de autobuze să își actualizeze cu ușurință serviciile și să asigure acuratețea.

Tema centrală a lucrării de diplomă este descrierea detaliată a componentelor serverului aplicației web, care asigură gestionarea fluidă a datelor utilizatorilor, a tranzacțiilor de achiziție a biletelor, a informațiilor de program și a urmăririi autobuzelor în timp real.

**Cuvinte cheie:** transport public, urmărire în timp real, aplicație web, Rails API

## **Kivonat**

The purpose of the web application is to make modern public transportation systems more user-friendly and efficient, with a particular focus on trip planning, ticket purchasing, and bus tracking. The application allows users to plan their trips in advance within a specific city or region's public transportation network. Users can purchase tickets for a selected route and view the departure and arrival times of the services, broken down by cities and stops, thus ensuring flexibility and accurate travel information.

The application also offers an interactive map that allows real-time tracking of the current positions of buses. This feature is especially useful when unexpected changes occur in the schedule, as users can be timely informed about delays or route changes. In addition to real-time tracking, the system provides the precise geographical locations of individual bus stops, which users can filter based on cities or routes, making it easier to plan which stop to arrive at.

A separate application running on the POS terminals located on the buses ensures ticket validation and the continuous sharing of the buses' positions with the central server. This feature ensures that all participants in the system have up-to-date information, allowing users to stay informed about the real-time locations of the buses.

Another important component of the application is the administration interface used by the bus companies. This interface allows for the management of schedules, the addition of new routes and stops, and the updating of existing data. Additionally, employee information, including details of drivers and other staff, as well as technical and operational data of the buses, can be managed here. This administration system ensures efficient operations and schedule maintenance, enabling bus companies to easily update their services and ensure accuracy.

The central theme of the thesis is the detailed description of the server-side components of the web application, which ensure the seamless management of user data, ticket purchase transactions, schedule information, and real-time bus tracking.

**Keywords:** public transportation, real-time tracking, web application, Rails API

## Kivonat

A webalkalmazás célja a modern tömegközlekedési rendszerek felhasználóbaráttá és hatékonyabbá tétele, különös tekintettel az utazás megtervezésére, jegyvásárlásra, valamint az autóbuszok követésére. Az alkalmazás lehetővé teszi a felhasználók számára, hogy előre megtervezhessék utazásaikat egy adott város vagy régió tömegközlekedési hálózatán belül. A felhasználók megvásárolhatják jegyeiket egy kiválasztott járatra, valamint megtekinthetik a járatok indulási és érkezési időpontjait, városokra és megállókra bontva, biztosítva ezzel a rugalmasságot és a pontos utazási információkat.

Az alkalmazás egy interaktív térképet is kínál, amelyen valós időben nyomon követhetők az autóbuszok jelenlegi pozíciói. Ez a funkció különösen hasznos, amikor a menetrendben váratlan változások történnek, mivel a felhasználók időben értesülhetnek a késésekről vagy útvonal-módosításokról. A valós idejű követés mellett a rendszer biztosítja az egyes buszmegállók pontos földrajzi helyzetét, amelyeket a felhasználók városok vagy útvonalak alapján szűrhetnek, így könnyebben tervezhetik meg, melyik megállóhoz érkezenek.

A buszokon található POS terminálokon futó különálló alkalmazás gondoskodik a jegyérvényesítésről és az autóbuszok pozíciójának folyamatos megosztásáról a központi szerverrel. Ez a funkció biztosítja, hogy a rendszer minden résztvevője naprakész információval rendelkezzen, így a felhasználók folyamatosan tájékozódhatnak az autóbuszok valós helyzetéről.

Az alkalmazás másik fontos része az adminisztrációs felület, amelyet a busztársaságok használhatnak. Ezen a felületen lehetőség van a menetrendek kezelésére, új útvonalak és megállók felvitelére, valamint a meglévő adatok frissítésére. Továbbá itt kezelhetők az alkalmazottak információi, beleértve a sofőrök és egyéb személyzet adatait, valamint az autóbuszok műszaki és üzemeltetési adatait. Ez az adminisztrációs rendszer biztosítja a hatékony működést és a menetrendek karbantartását, így a busztársaságok könnyedén frissíthetik szolgáltatásaikat és biztosíthatják a pontosságot.

A dolgozat központi témája a webalkalmazás szerver oldali komponenseinek részletes ismertetése, amelyek biztosítják a felhasználói adatok, a jegyvásárlási tranzakciók, a menetrendi információk, valamint a valós idejű autóbusz-követés zökkenőmentes kezelését.

**Kulcsszavak:** tömegközlekedés, valós idejű követés, webalkalmazás, Rails API

# Tartalomjegyzék

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>1. Bevezető</b>                                        | <b>4</b>  |
| <b>2. Célkitűzések</b>                                    | <b>5</b>  |
| <b>3. Elméleti megalapozás és bibliográfiai tanulmány</b> | <b>6</b>  |
| 3.1. Technológiai háttér . . . . .                        | 6         |
| 3.1.1. Ruby on Rails . . . . .                            | 6         |
| 3.1.2. PostgreSQL . . . . .                               | 7         |
| 3.1.3. Azure . . . . .                                    | 7         |
| 3.2. Integráció és fejlesztési lehetőségek . . . . .      | 7         |
| 3.3. Fenntarthatóság . . . . .                            | 8         |
| 3.4. Hasonló alkalmazások . . . . .                       | 8         |
| 3.4.1. Felhasználói élmény és innovációk . . . . .        | 8         |
| 3.4.2. Hatások a közlekedési rendszerre . . . . .         | 9         |
| <b>4. Rendszer specifikációi</b>                          | <b>10</b> |
| 4.1. Felhasználói követelmények . . . . .                 | 10        |
| 4.1.1. Felhasználói weboldal . . . . .                    | 10        |
| 4.1.2. Admin felület . . . . .                            | 10        |
| 4.1.3. Mobil applikáció . . . . .                         | 11        |
| 4.1.4. POS terminál alkalmazás . . . . .                  | 11        |
| 4.2. Rendszerkövetelmények . . . . .                      | 12        |
| 4.2.1. Funkcionális követelmények . . . . .               | 12        |
| 4.2.2. Nem-funkcionális követelmények . . . . .           | 13        |
| <b>5. A rendszer részletes leírása</b>                    | <b>15</b> |
| 5.1. Az API felépítése . . . . .                          | 16        |
| 5.2. Endpointok részletes leírása . . . . .               | 16        |
| 5.2.1. GET /get_stations_by_city . . . . .                | 16        |
| 5.3. Adatkezelés és adatbázis kapcsolat . . . . .         | 17        |
| 5.3.1. Adatbázis táblák . . . . .                         | 17        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>6. Infrastruktúra</b>                              | <b>20</b> |
| 6.1. Azure infrastruktúra áttekintése . . . . .       | 20        |
| 6.2. Domain konfiguráció . . . . .                    | 20        |
| 6.3. Adatbázis telepítése és konfigurációja . . . . . | 20        |
| 6.4. Szerver telepítése . . . . .                     | 21        |
| <b>7. A rendszer tesztelése</b>                       | <b>22</b> |
| 7.1. Tesztelési környezet . . . . .                   | 22        |
| 7.2. Funkcionális tesztelés . . . . .                 | 22        |
| 7.2.1. Végpontok tesztelése . . . . .                 | 23        |
| 7.2.2. Tesztelési példa . . . . .                     | 23        |
| 7.2.3. Tesztelési eredmények . . . . .                | 24        |
| 7.3. Teljesítménytesztelés . . . . .                  | 24        |
| 7.3.1. Teljesítmény-metrikák . . . . .                | 24        |
| 7.4. Biztonsági tesztelés . . . . .                   | 24        |
| 7.5. Összegzés . . . . .                              | 25        |
| <b>8. Összefoglalás</b>                               | <b>26</b> |
| <b>Irodalomjegyzék</b>                                | <b>27</b> |
| <b>A Függelék</b>                                     | <b>29</b> |

# 1. fejezet

## Bevezető

A mai világunk velejárója, hogy egyre több dolgot digitalizálunk az életünk egyszerűsítésének érdekében. Az autóbuszok használata a közlekedés egyik legkörnyezetkímélőbb formája, de sajnos sok hátránnyal is jár. Ezt otthon is megfigyeltük, de amióta egyetemre járunk, itt Marosvásárhelyen is megtapasztaltuk a tömegközlekedés hiányosságait. Ezek közül a legkiemelkedőbb a kártyás fizetés lehetőségének a hiánya. Újonnan a városi buszokon adódik már erre lehetőség, ám a hosszútávú utazásoknál még mindig csak készpénzzel lehet fizetni.

Jelenleg arra sincs lehetőség, hogy egy utazásra előre jegyet vásároljunk. Egy másik nagy probléma, hogy nem minden megállóban elérhető a buszok közlekedésének menetrendje, illetve ezek online sehol nem tekinthetők meg. Emellett Vásárhelyen a legnagyobb problémának a pontos közlekedés hiánya tekinthető szerintünk. Ha egy adott megállóban ki is van írva, hogy mikor közlekednek a buszok, sok esetben vagy késnek, vagy rosszabb esetben hamarabb elindulnak. Kiaknázva a technológia által nyújtott lehetőségeket, szeretnénk egy olyan alkalmazást létrehozni, amely segíti a tömegközlekedés népszerűsítését, és egyszerűbbé teszi a buszok használatát.

Busztársaságok szempontjából is fontos a digitalizáció. A profitot és a költségeket egyszerűbb és átláthatóbb egy online térben számon tartani, mint papíron. A buszok állapotát, papírjainak lejáratát is hasznos online követni, mivel így elkerülhetők az esetleges büntetések.

Úgy gondoljuk, hogy sok embert tudnánk motiválni a tömegközlekedés használatára, ha a fenti problémákat egy alkalmazással meg tudnánk oldani, javítva a busztársaságok, és a felhasználók élményét is.

Emellett kiemelten fontosnak tartjuk a környezetvédelmet is, hiszen a buszközlekedés támogatása hozzájárul a fenntartható városi közlekedéshez. A digitális megoldások alkalmazásával nemcsak a felhasználói élményt javíthatjuk, hanem hozzájárulhatunk a városok környezetbarát fejlődéséhez is.



## 2. fejezet

# Célkitűzések

A projekt célja, hogy egy modern, felhasználóbarát webalkalmazást fejlesszen ki a tömegközlekedés népszerűsítésére és a buszok használatának egyszerűsítésére. A következő konkrét célkitűzéseket határozzuk meg:

- **Digitális Jegyvásárlás:** Olyan funkció kialakítása, amely lehetővé teszi a felhasználók számára, hogy előre jegyet vásároljanak a kívánt járatra, így csökkentve a készpénzes tranzakciók számát.
- **Valós Idejű Buszkövetés:** Integrálni egy valós idejű buszkövető rendszert, amely lehetővé teszi a felhasználók számára, hogy nyomon követhessék a buszok aktuális helyzetét és várható érkezési idejét.
- **Menetrendek Elérhetősége:** Biztosítani, hogy a buszmenetrendek minden megállóban könnyen elérhetők legyenek, valamint online platformon is megtekinthetők legyenek.
- **Interaktív Térkép:** Fejleszteni egy interaktív térképet, amely lehetővé teszi a felhasználók számára, hogy könnyen navigáljanak a város tömegközlekedési hálózatán, és információkat kapjanak a megállókról és járatokról.
- **Adminisztrációs Felület:** Létrehozni egy adminisztrációs felületet, amelyet a busztársaságok használhatnak a menetrendek, útvonalak és alkalmazottak információinak kezelésére, ezzel növelve a működés hatékonyságát.
- **Felhasználói Élmény Javítása:** A webalkalmazás tervezése során a felhasználói élmény maximalizálása, beleértve a könnyű navigációt és az intuitív felület kialakítását.

Ezek a célkitűzések a webalkalmazás fejlesztésének alapját képezik, és hozzájárulnak a modern tömegközlekedési rendszerek hatékonyabbá tételéhez, valamint a felhasználók utazási élményének javításához.

## 3. fejezet

# Elméleti megalapozás és bibliográfiai tanulmány

### 3.1. Technológiai háttér

A Bus4U alkalmazás digitális megoldásai mögött modern webtechnológiák állnak, amelyek lehetővé teszik a felhasználói élmény folyamatos fejlesztését és a szolgáltatások hatékony működését. Az alkalmazás backendje Ruby on Rails keretrendszert használ, amely az egyik legnépszerűbb választás webes alkalmazások fejlesztésére.

#### 3.1.1. Ruby on Rails

A Ruby on Rails egy *nyílt forráskódú keretrendszer*, amely gyors fejlesztést és egyszerű karbantartást tesz lehetővé a „*Convention over Configuration*” (Konvenció a konfiguráció felett) és a „*Don't Repeat Yourself*” (Ne ismételd önmagad) elvek alkalmazásával. Ezek az elvek elősegítik, hogy a fejlesztés kevesebb kóddal történjen, így könnyebb lesz a karbantartás, valamint az új funkciók beépítése is. Az *MVC (Model-View-Controller)* architektúra révén a Ruby on Rails jól strukturált rendszert biztosít, amelyben az üzleti logika, az adatkezelés és a felhasználói felület egymástól elkülönítve, de összhangban működik.

A Ruby on Rails további előnyei:

- **Nagy és aktív közösség:** rengeteg dokumentáció, könyvtár (gem) és integráció áll rendelkezésre, ami gyorsabbá és biztonságosabbá teszi a fejlesztést.
- **Automatizált tesztelés:** A Rails támogatja az automatizált tesztelést, amely a megbízhatóságot növeli.
- **RESTful struktúra:** A Rails natív módon támogatja a RESTful API-k kialakítását, ami megkönnyíti az API-k integrációját más rendszerekkel és frontendekkel.

### 3.1.2. PostgreSQL

A *PostgreSQL* egy *nyílt forráskódú, objektum-relációs adatbázis-kezelő rendszer*, amely gazdag szolgáltatáskészletet és rugalmasságot biztosít, így ideális választás olyan alkalmazások számára, amelyek összetett adatstruktúrákkal és nagyméretű adattömegekkel dolgoznak.

A PostgreSQL előnyei a Bus4U számára:

- **Teljesítmény optimalizálása és bonyolult lekérdezések kezelése:** PostgreSQL kiválóan kezeli a párhuzamos műveleteket és a nagy adattömegeket, így biztosítva az alkalmazás gyors működését.
- **Skálázhatóság:** nagyobb adatmennyiség vagy felhasználói terhelés esetén a PostgreSQL támogatja a függőleges és vízszintes skálázást, ezáltal hosszú távon fenntartható teljesítményt biztosít.

### 3.1.3. Azure

Az *Azure* egy nagyvállalati szintű, biztonságos és skálázható felhőplatform, amely biztosítja a Bus4U alkalmazás számára a megbízható működést. Az Azure környezet használatával az infrastruktúra rugalmasan kezelhető, és az erőforrások felhasználása hatékonyan szabályozható.

Az Azure előnyei a Bus4U számára:

- **Skálázhatóság és rugalmasság:** Az Azure képes automatikusan alkalmazkodni a terheléshez, így ha megnövekszik a felhasználói forgalom, a rendszer könnyedén alkalmazkodik.
- **Biztonság és adatvédelem:** Az Azure szigorú adatvédelmi szabályokat és fejlett biztonsági protokollokat biztosít, amelyek elengedhetetlenek egy tömegközlekedési alkalmazás számára.

## 3.2. Integráció és fejlesztési lehetőségek

A Ruby on Rails, PostgreSQL és Azure kombinációja lehetővé teszi számunkra, hogy gyorsan új funkciókat integráljunk. A RESTful API-k használata révén a különböző komponensek (például a mobilalkalmazás és a backend) zökkenőmentesen kommunikálnak egymással, javítva az alkalmazás teljesítményét.

### 3.3. Fenntarthatóság

A Bus4U projekt nemcsak a közlekedési információk egyszerűsítésére szolgál, hanem fontos szerepet játszik a fenntartható közlekedés népszerűsítésében is. Az online jegyvásárlás és az alternatív közlekedési módok ösztönzése hozzájárul a közlekedési rendszer környezeti terheinek csökkentéséhez.

### 3.4. Hasonló alkalmazások

A közlekedési alkalmazások, mint például a Budapest Go, jelentős hatással vannak a modern városi közlekedésre. A Budapest Go a magyarországi főváros tömegközlekedési rendszerének digitális megoldása, amely lehetővé teszi a felhasználók számára, hogy egyszerűen és gyorsan intézzék utazásaik szervezését. Az alkalmazás számos funkcióval rendelkezik, amelyek javítják a felhasználói élményt.

A Budapest Go célja, hogy a város tömegközlekedését még hozzáférhetőbbé és felhasználóbarátabbá tegye. Az alkalmazás főbb jellemzői közé tartozik:

- Valós idejű menetrendek: Az alkalmazás folyamatosan frissíti a buszok, villamosok és metrók érkezési idejét, lehetővé téve a felhasználók számára, hogy pontosan tudják, mikor indul a következő járat.
- Online jegyvásárlás: A felhasználók az alkalmazáson belül jegyeket vásárolhatnak, ami gyorsítja a folyamatot és csökkenti a sorban állás szükségességét.
- Útvonaltervezés: Az alkalmazás lehetővé teszi a felhasználók számára, hogy megtervezzék utazásaikat, figyelembe véve a különböző közlekedési eszközöket és a lehető leghatékonyabb útvonalakat.
- Használati statisztikák: A Budapest Go elemzi a felhasználók utazási szokásait, lehetővé téve a szolgáltatók számára, hogy jobban megértsék a közlekedési igényeket és fejleszthessék szolgáltatásaikat.

#### 3.4.1. Felhasználói élmény és innovációk

A Budapest Go nemcsak a közlekedési információk egyszerűsítésére szolgál, hanem innovatív megoldásokat is kínál a felhasználók számára. Az alkalmazás intuitív felhasználói felülete lehetővé teszi a könnyű navigációt, így még a technológiához kevésbé értő felhasználók is könnyen el tudnak igazodni.

### **3.4.2. Hatások a közlekedési rendszerre**

A Budapest Go alkalmazás bevezetése jelentős hatással volt a főváros közlekedési rendszerére. A digitális jegyvásárlás elterjedése csökkentette a készpénzes tranzakciók számát, így gyorsabbá téve a közlekedési folyamatokat. Ezenkívül a valós idejű információk elérhetősége segít a felhasználóknak a jobb döntéshozatalban, ami hozzájárul a város közlekedési hatékonyságának növeléséhez.

A Budapest Go egy példa arra, hogyan lehet a digitális technológiát alkalmazni a városi közlekedés fejlesztésére. Az alkalmazás által kínált funkciók és a felhasználói élmény folyamatos javítása hozzájárul a közlekedési rendszerek modernizálásához és a fenntartható közlekedés népszerűsítéséhez.

## 4. fejezet

# Rendszer specifikációi

### 4.1. Felhasználói követelmények

#### 4.1.1. Felhasználói weboldal

A felhasználói weboldal bejelentkezés nélkül használható. A főoldalon lehetőség nyílik járatokra keresni dátum, idő és cél alapján. A menetrendeknél megtekinthetők a buszok és megállók órarendjei. A megállók oldalán az összes megálló térképen jelenik meg, és szűrést is lehet alkalmazni. Az élőben frissülő térképen a buszok pozícióját percről percre lehet követni. A jegyek oldalán listázva vannak az eddig megvásárolt jegyek QR-kódjai.

Fiókot létrehozni kizárólag a jegyvásárláshoz szükséges. A főoldalon, ha a felhasználó megtalálta a megfelelő járatot, ráléphet a vásárlás gombra. A fiók regisztrációjához meg kell adni:

- teljes név
- email cím
- új jelszó
- ellenőrző kód (amit a rendszer elküld a megadott email címre)

#### 4.1.2. Admin felület

Az admin felület kizárólag bejelentkezéssel elérhető. Új fiókokat csak az adott cég vezetője tud létrehozni az "Alkalmazottak" oldalon. Az adminoknak lehetőségük van:

- a megállók listájának szerkesztésére
- az útvonalak összes tulajdonságának módosítására (meglátogatott megállók, indulási időpontok, jegyárak)

- a főoldalon elérhető statisztikák megtekintésére (eladott jegyek száma, megtett kilométer, bevétel)
- a buszok adatainak kezelésére (útadó, biztosítás, műszaki vizsga, buszok kivonása forgalomból, új buszok hozzáadása)

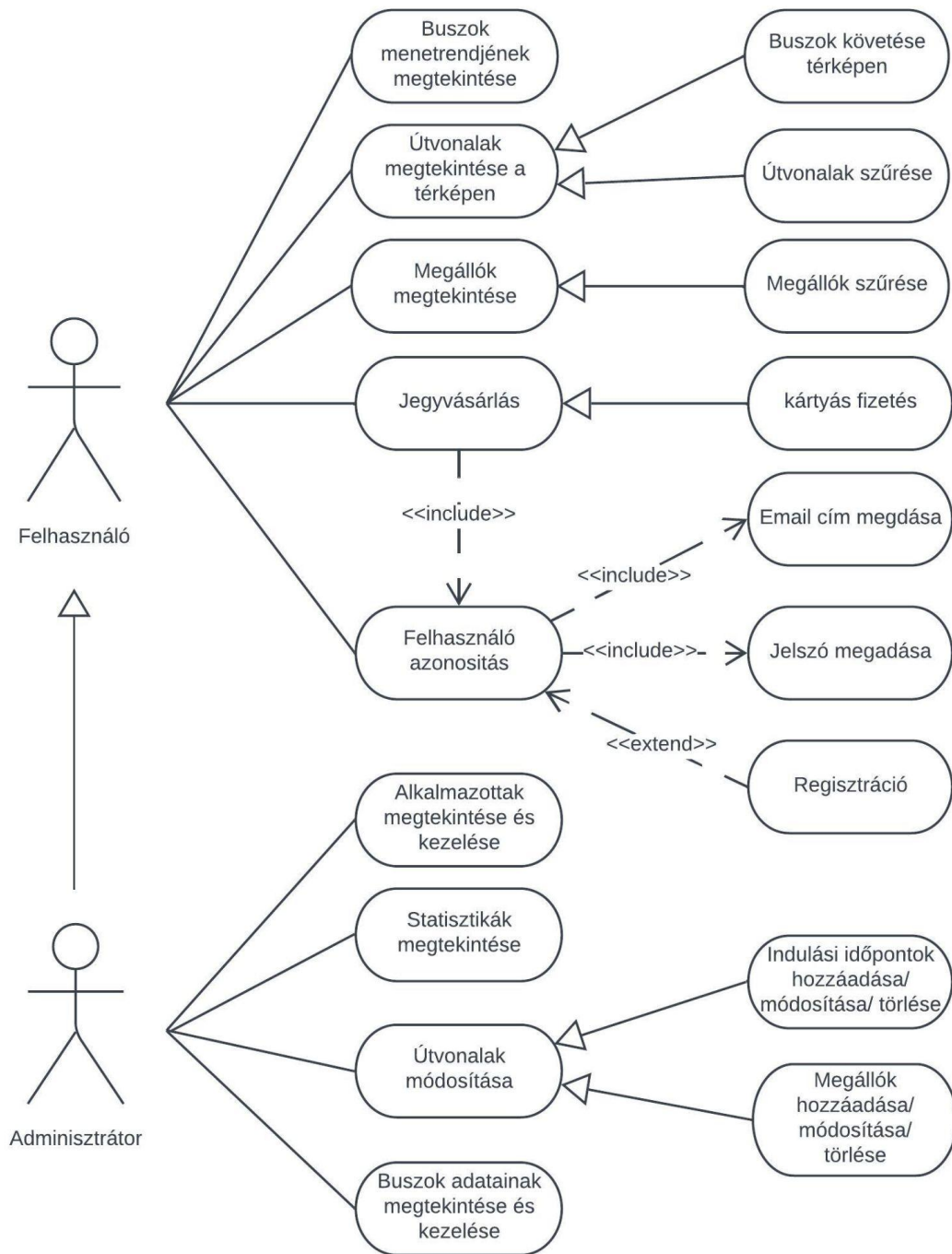
#### **4.1.3. Mobil applikáció**

A Flutter alapú mobil applikáció ugyanúgy működik, mint a felhasználóknak szánt weboldal, így ugyanazok a követelmények vonatkoznak erre is.

#### **4.1.4. POS terminál alkalmazás**

A POS terminál alkalmazás használatához szükség van egy sofőr szerepkörű admin fiókra. Bejelentkezés után lehetőség van egy adott busz követésének elindítására, valamint a jegyszkennelés működtetésére.

A rendszer használati eset diagramja az 4.1-es ábrán látható.



4.1. ábra. Use-case diagram a felhasználók és az adminisztrátorok lehetőségeiről

## 4.2. Rendszerkövetelmények

### 4.2.1. Funkcionális követelmények

A felhasználói felület minden oldalán van egy kis form, amelyet kitöltve le lehet kérni az adatbázisból az adott oldalon megjelenítendő adatokat. Ilyenre példa van a főoldal-



lon is, ahol meg kell adni az indulás helyét, a cél helyét (települést) és opcionálisan az ezekhez tartozó megállót, egy-egy lenyíló listából kiválasztva az adatokat. A települések betöltődnek az oldallal együtt, a megállók listája viszont csak a város kiválasztása után, mindig csak az adott településen találhatóak jelennek meg. Ezen kívül a főoldalon van egy dátumválasztó és egy időválasztó is, ezek alapértelmezetten az aktuális dátummal és idővel vannak kitöltve. Ha a form ki van töltve, a keresés gomb lenyomására az oldal elküldi a bevitt adatokat a backend szerverre, amely válaszként elküld egy listát a megfelelő utazási lehetőségekkel. Ezek az adatok JSON formátumban, HTTP protokollon keresztül közlekednek a frontend és backend között. A többi oldal is hasonlóan működik, először az adatokat kell megadni, aztán a gomb lenyomására pillanatok alatt megjelenik a képernyőn a szerver válasza a bevitt adatokra.

A különböző formokon kívül, a weboldalon található térképek is API hívásokkal működnek. A rendszer először betölti az alap térképet a MapBox API segítségével, aztán amint a felhasználó kiválasztotta a megtekinteni kívánt útvonalat a Menetrend vagy az Élő térkép lapon, a weboldal a saját adatbázisunkból kéri le a hozzá tartozó megállók koordinátáit, ezeket megjeleníti a térképen, végül ismét a MapBox API-t használva útvonaltervet generáltat. Ez úgy működik, hogy a rendszer elküldi a megállók listáját és visszakap egy sok pontból álló új listát a MapBox-tól, ezeket összekötve sok kis vonallal, kirajzolódik a részletes útvonal a térképen.

#### 4.2.2. Nem-funkcionális követelmények

##### Szervezéssel kapcsolatos követelmények ♦

- **Programozási nyelvek:** Ruby, Dart, HTML, CSS, JavaScript
- **Keretrendszerek:** Ruby on Rails, Flutter, Vue.js
- **IDE:** Visual Studio Code
- **Verziókövetés:** Git és GitHub
- **Adatbázis:** PostgreSQL
- **Cloud szolgáltatás:** Azure
- **Webhoszt:** Netlify
- **Csoportos kommunikáció:** Slack

**Termékkel kapcsolatos követelmények** ♦ A felhasználói weboldal és az admin felület használatához modern, internetezésre alkalmas eszközre van szükség, legalább 2018-as vagy frissebb böngészővel:

- **Chrome:** 87 vagy újabb
- **Firefox:** 78 vagy újabb
- **Safari:** 14 vagy újabb
- **Edge:** 88 vagy újabb

Az applikáció használatához internettel és GPS antennával rendelkező okostelefon vagy tablet szükséges, a következő minimális követelményekkel:

- **Operációs rendszer:** Android 5.0 vagy iOS 8.0 és újabb
- **Szabad tárhely:** minimum 120 MB
- **Memória:** 516–1024 MB

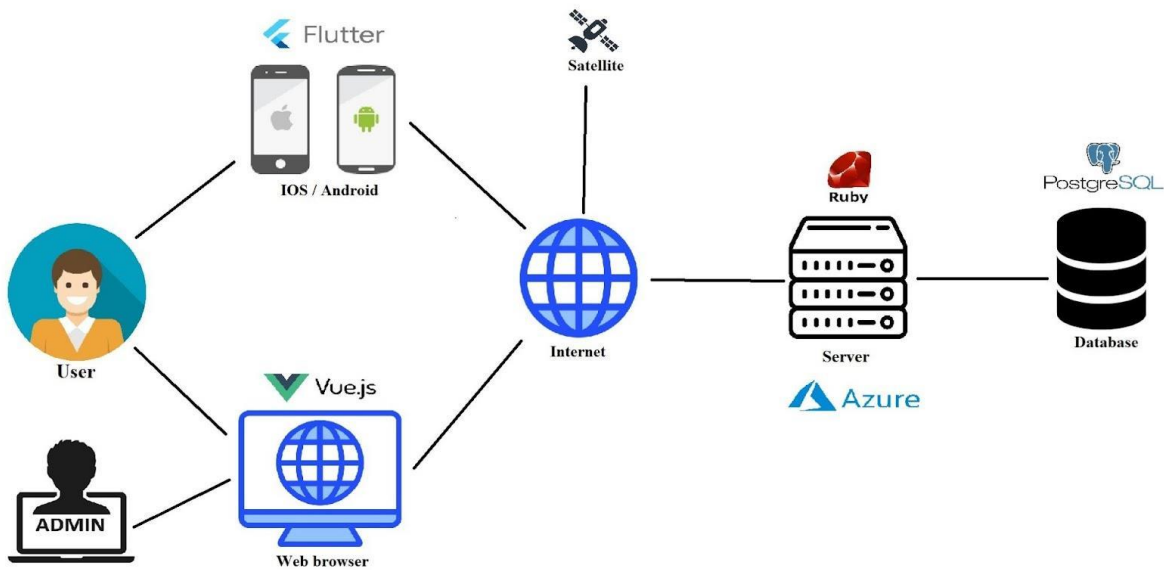
A POS terminál alkalmazás futtatásához egy Sunmi V2 PRO POS terminál ajánlott. Alternatívaként, ha nem áll rendelkezésre, bármely kamerával és GPS antennával ellátott, minimum Android 5.0 vagy iOS 8-at futtató eszköz is használható:

- **Szabad tárhely:** legalább 100 MB
- **Memória:** minimum 510 MB

## 5. fejezet

# A rendszer részletes leírása

Rendszerünk öt fő komponensből áll, amelyeket a komponens diagramon is láthatunk: egy Ruby on Rails keretrendszerben írt API-ból, egy Vue.js alapú webes felhasználói- és admin felületből, egy Flutter alkalmazásból a felhasználók számára, valamint egy Flutter keretrendszerben készült POS terminál alkalmazásból. Az API a REST architektúrát követi, lehetővé téve a különböző erőforrások, például buszok, járatok és jegyek kezelését. A felhasználói és admin weboldal Vue-ban készült, a Vuetify elemek felhasználásával, ami lehetővé tette az egyszerű és hatékony felhasználói felületek gyors fejlesztését. A Vue.js keretrendszer előnye, hogy komponensekből áll, így átlátható és jól rendszerezett webalkalmazás felépítést tesz lehetővé. Ezen kívül a Vuetify komponens-készlete megkönnyítette az UI elemek létrehozását, gyorsítva ezzel a fejlesztési folyamatot. Az alkalmazás Flutterben készült, Dart programozási nyelvet használva, amely lehetővé teszi, hogy ugyanazt a kódbázist használjuk iOS és Android platformokra is, így jelentősen csökkentve a fejlesztési időt és költségeket. A POS terminál alkalmazása is Flutter keretrendszerben készült, biztosítva a hatékony és egységes megoldást a különböző platformok számára. A Vue.js alapú weboldalakat, illetve a felhasználóknak szánt Flutter alkalmazást Portik Szabolcs kollegám mutatja be részletesebben egy másik dolgozat keretein belül.



5.1. ábra. A rendszer komponensdiagramja

## 5.1. Az API felépítése

A Bus4U API a REST architektúrát követi, és számos végpontot biztosít a különböző erőforrások kezeléséhez, mint például:

- **Buszok kezelése:** buszok létrehozása, frissítése, törlése.
- **Járatok menedzselése:** járatok lekérése és ütemezése.
- **Jegyek és bérletek:** jegyek és bérletek vásárlása és kezelése.

## 5.2. Endpointok részletes leírása

### 5.2.1. GET /get\_stations\_by\_city

Ez az endpoint lehetővé teszi a megállók listájának lekérését egy adott város alapján. A kérés a következő paramétert használja:

- **city\_uid:** A város egyedi azonosítója.

Példa kérés:

GET [https://api.bus4u.online/v1/get\\_stations\\_by\\_city?city\\_uid=CTY\\_N11XH](https://api.bus4u.online/v1/get_stations_by_city?city_uid=CTY_N11XH)

Válasz formátum:

```

{
  "stations": [
    {
      "station_uid": "STT_EGAVX",
      "name": "Sapientia",
      "address": "Str. Calea Sighisoarei nr. 2",
      "longitude": "24.59883",
      "latitude": "46.52339"
    },
    {
      "station_uid": "STT_6C03Q",
      "name": "Dedeman",
      "address": "Bul. 1 Decembrie nr. 189",
      "longitude": "24.59941",
      "latitude": "46.52678"
    }
  ]
}

```

## 5.3. Adatkezelés és adatbázis kapcsolat

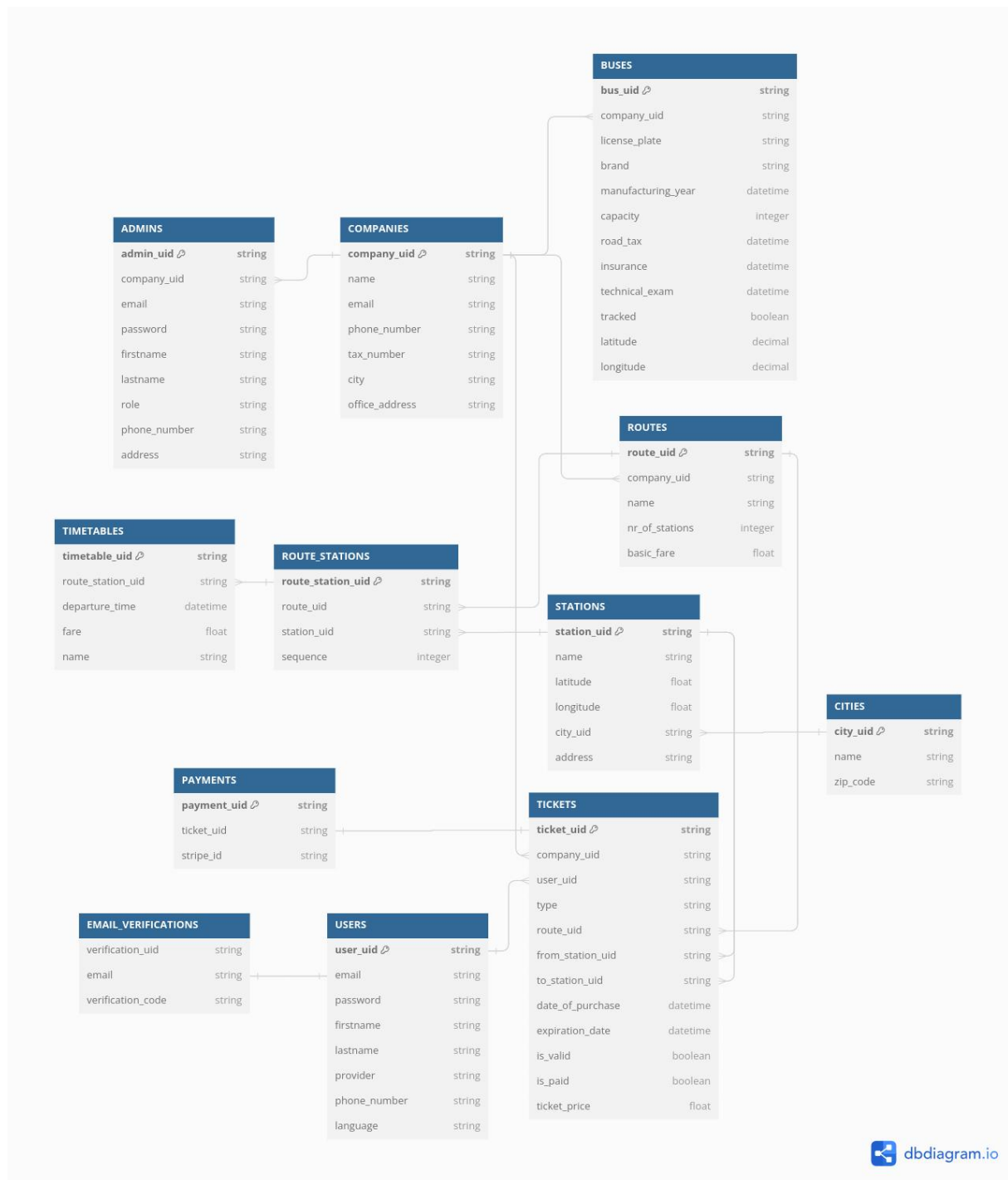
Az API végpontjai a **PostgreSQL** adatbázissal kommunikálnak. Az adatbázis tranzakciókat az **ActiveRecord** ORM segítségével kezeljük, amely lehetővé teszi az adatok egyszerű és hatékony lekérését, beszúrását és frissítését.

### 5.3.1. Adatbázis táblák

A PostgreSQL adatbázisunk a következő táblákból áll:

- **Companies**: Azon cégek adatai, akikkel szerződést kötött a Bus4U.
- **Admins**: Különböző cégekhez tartozó adminisztrátorok adatai. Szerepük lehet “boss” vagy “driver”.
- **Buses**: A cégekhez tartozó autóbuszok adatai.
- **Routes**: A cégek saját útvonalai.
- **Stations**: A rendszerben található buszmegállók összessége.
- **Route\_stations**: Egy cég adott útvonalán keresztülhaladó megállók.

- **Timetables**: Egy útvonal megállójának indulási időpontjai és ára nap szerinti felosztásban.
- **Cities**: Az összes város, amelyen áthaladnak útvonalak.
- **Users**: A Bus4U felhasználói a fontosabb adataikkal.
- **Email\_verifications**: Az adott email címekhez tartozó regisztrációkor felhasználható kódok.
- **Tickets**: A felhasználókhöz tartozó jegyek.
- **Payments**: Stripe fizetések eltárolása.



5.2. ábra. Az rendszer adatbázisdiagramja

## 6. fejezet

# Infrastruktúra

### 6.1. Azure infrastruktúra áttekintése

A Bus4U rendszer a Microsoft Azure felhőalapú platformján került telepítésre. Az Azure szolgáltatások skálázhatóságot és megbízhatóságot biztosítanak a rendszer számára. A telepítéshez az alábbi főbb szolgáltatásokat használtam:

- **Azure Virtual Machines (VM):** A szerver hosztolása.
- **Azure Database for PostgreSQL:** Az adatbázis-szolgáltatás biztosítása.

### 6.2. Domain konfiguráció

A rendszer az **api.bus4u.online** domainen érhető el, amelyet a Cloudflare platformján konfiguráltam. A domain hozzárendelése során a virtuális gép (VM) IP címét használtam. A következő beállításokat végeztem el:

- **DNS beállítások:** A Cloudflare-en elvégeztem a domain névhez tartozó A és CNAME rekordok konfigurálását.
- **SSL tanúsítványok:** A HTTPS alapú kapcsolat biztonságának érdekében a Cloudflare szolgáltatásait alkalmazzuk, amelyek védelmet nyújtanak a felhasználók számára.

### 6.3. Adatbázis telepítése és konfigurációja

A rendszer adatbázisa egy **Azure Database for PostgreSQL** szolgáltatáson keresztül került telepítésre, amely lehetővé teszi az adatbázis automatikus méretezését és a folyamatos biztonsági mentések készítését. A következő beállításokat alkalmazzuk:

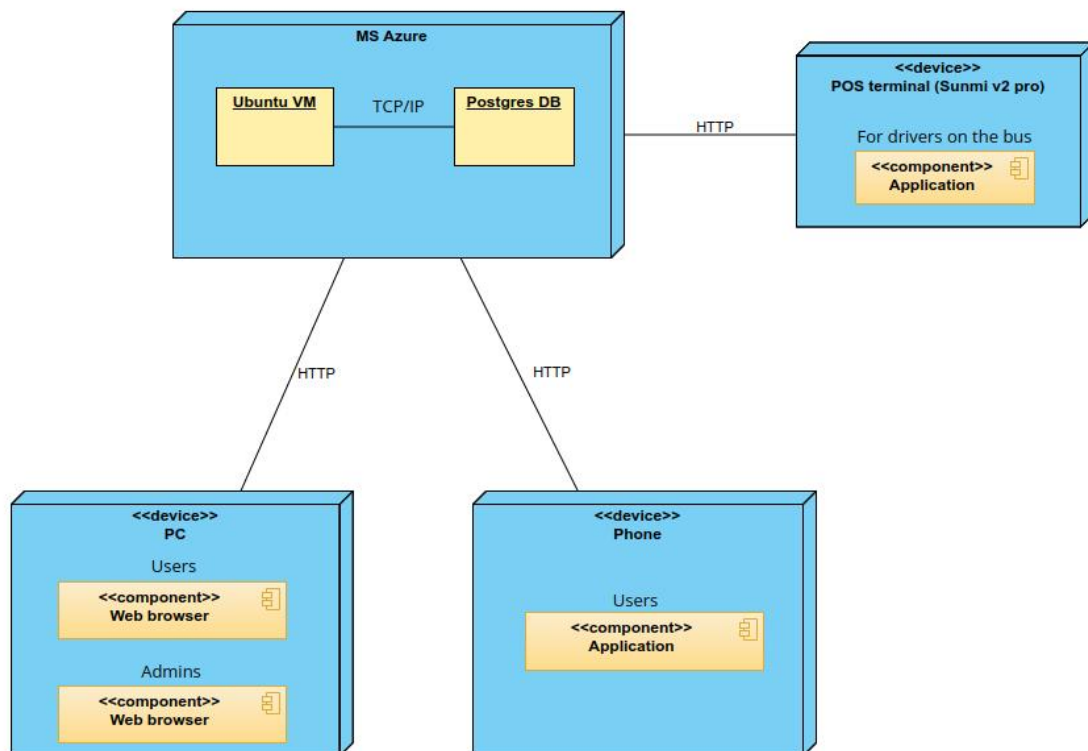


- **Titkosított kapcsolatok:** A PostgreSQL adatbázis SSL tanúsítványokat használ a biztonságos kommunikációhoz.
- **Backup és recovery:** Automatikus biztonsági mentések kerülnek tárolásra az adatvesztés elkerülése érdekében.

## 6.4. Szerver telepítése

A szerver az **Azure Virtual Machines** szolgáltatásban került telepítésre, ahol a Ruby on Rails alapú API-t futtatjuk. A telepítés során a következő lépéseket hajtottuk végre:

- **Környezeti változók beállítása:** A szükséges konfigurációs változók (pl. adatbázis kapcsolatok) a Ruby on Rails beépített credentials funkciójában lettek biztonságosan tárolva és kezelve.
- **Verziókezelés:** Az **asdf** verziókezelő használatával a Ruby és egyéb nyelvek verzióit könnyen kezelhetjük, ami megkönnyíti a fejlesztési környezet fenntartását.
- **CI/CD pipeline:** Az alkalmazás automatikus telepítése és frissítése a GitHub Actions integrációjával valósult meg.



6.1. ábra. Telepítési diagram

## 7. fejezet

# A rendszer tesztelése

### 7.1. Tesztelési környezet

A tesztelést egy helyi fejlesztési környezetben végeztem. A szerver a következő konfigurációval működött:

- **Operációs rendszer:** Ubuntu 22.04 LTS
- **Szerver:** Ruby on Rails 7.0.4
- **Adatbázis:** PostgreSQL 14
- **Tesztelő eszköz:** Postman v10.0

### 7.2. Funkcionális tesztelés

A rendszer funkcionális teszteléséhez a Postman eszközt használtam. A Postman lehetőséget nyújt HTTP-kérések küldésére, valamint a különböző API végpontok működésének, válaszainak és adatstruktúráinak ellenőrzésére. Az alábbiakban bemutatom a tesztelés folyamatát és az eredményeket.

A tesztelést egy helyi fejlesztési környezetben végeztem, ahol a szerver a következő címen volt elérhető:

`http://localhost:3000/api/v1/`

A Postman segítségével különböző HTTP metódusokat (GET, POST, PUT, DELETE) használtam a végpontok tesztelésére. Az egyes végpontokat a rendszer megfelelő válaszai alapján ellenőriztem.

### 7.2.1. Végpontok tesztelése

Az alábbi végpontok tesztelését szeretném kiemelni:

- **GET** `api_url/v1/get_stations_by_city` – Lekéri az állomásokat a megadott város alapján.
- **GET** `/v1/get_departure_times_for_station_in_route` – Indulási időpontok lekérése egy adott járatra és állomásra.
- **GET** `/v1/get_available_tickets` – Elérhető jegyek lekérdezése az adott kiindulási és célállomásra.
- **GET** `/v1/get_bus_locations_by_route` – A buszok aktuális helyzetének lekérése egy adott járatra.
- **GET** `/v1/tickets_of_user` – Az aktuális felhasználó jegyeinek lekérése.
- **GET** `/v1/admin/document_validity_checker` – Dokumentum érvényességének ellenőrzése adminisztrátorok számára.
- **GET** `/v1/admin/statistics` – Statisztikai adatok lekérdezése egy adott cég számára.

### 7.2.2. Tesztelési példa

Például a GET metódus használatával kértem le az állomásokat egy megadott város alapján:

```
GET api.bus4u.online/v1/get_stations_by_city?city_uid=CTY_N11XH
```

A szerver válasza a következő JSON formátumban érkezett:

```
{
  "stations": [
    {
      "station_uid": "STT_EGAVX",
      "name": "Sapientia",
      "address": "Str. Calea Sighisoarei nr. 2",
      "longitude": "24.59883",
      "latitude": "46.52339"
    },
    {
      "station_uid": "STT_6C03Q",
```

```

        "name": "Dedeman",
        "address": "Bul. 1 Decembrie nr. 189",
        "longitude": "24.59941",
        "latitude": "46.52678"
    }
]
}

```

Ezzel a lekérdezéssel sikeresen megkaptam az adatokat, amelyeket a rendszer helyesen dolgozott fel.

### 7.2.3. Tesztelési eredmények

A tesztelés során az összes végpontot felvezettem Postman-re, és mindegyik megfelelően működött. Az egyes végpontok válaszai megfelelték az elvárt adatstruktúrának, és a rendszer helyesen kezelte a kéréseket.

## 7.3. Teljesítménytesztelés

A teljesítménytesztelés célja a rendszer stabilitásának és skálázhatóságának értékelése volt nagy terhelés alatt. A vizsgálat során a JMeter eszközt használtam, amely képes különböző mennyiségű egyidejű felhasználói kérések szimulálására.

A JMeter mintavételi eredményei alapján az alábbi teljesítménymetrikák és azok jelentőségeit vizsgáljuk az API teljesítményével kapcsolatban:

### 7.3.1. Teljesítmény-metrikák

KIEGÉSZÍTENI AZ ÚJRATESZTELT ADATOKKAL

## 7.4. Biztonsági tesztelés

A biztonsági tesztelés során különös figyelmet fordítottam a következő sebezhetőségek elkerülésére:

- **SQL injection** – Input validálás és előkészített SQL lekérdezések használata.
- **Cross-site scripting (XSS)** – HTML entitások kódolása a felhasználói bemeneleknél.
- **Cross-site request forgery (CSRF)** – CSRF tokenek használata minden kényes műveletnél.

## 7.5. Összegzés

A tesztelési eredmények alapján a rendszer minden funkciója megfelelően működik, a teljesítménye kielégítő, és a biztonsági mechanizmusok jól védik az API-t a lehetséges támadások ellen. A további tesztelések célja az API skálázhatóságának javítása, valamint az integrációs lehetőségek vizsgálata más külső rendszerekkel.

## 8. fejezet

# Összefoglalás

Ez a fejezet összefoglalja a Bus4U rendszer API-jának tervezését, megvalósítását és tesztelését, valamint bemutatja a legfontosabb megállapításokat és javaslatokat a jövőbeli fejlesztésekhez.

A dolgozat célja egy hatékony busz menedzselő rendszer API létrehozása volt, amely lehetővé teszi a felhasználók számára a buszjáratok, jegyek és bérletek kezelését, valamint a rendszeres frissítéseket és karbantartásokat. A projekt során a következő lépéseket tettük:

1. Követelmények és tervezés: A rendszer funkcionális és nem funkcionális követelményeinek részletes elemzése és a legfontosabb funkciók prioritásának meghatározása. A követelmények alapján a rendszer architektúráját és az API felépítését terveztük meg.

2. API megvalósítása: Az API endpointok megvalósítása során külön figyelmet fordítottunk a RESTful architektúra elveinek betartására, biztosítva ezzel a könnyű használhatóságot és a skálázhatóságot. A végpontok különböző erőforrásokat (buszok, járatok, jegyek, bérletek) kezelnek, lehetővé téve a felhasználók számára, hogy egyszerűen hozzáférjenek az információkhoz és végrehajtsanak műveleteket.

3. Tesztelés: A rendszer tesztelését Postman segítségével végeztük, ahol különböző kéréseket küldtünk az API végpontokhoz, hogy ellenőrizzük azok működését, a válaszok helyességét és a teljesítményt. A tesztelési folyamat során sikerült azonosítani és javítani a hibákat, így a rendszer stabil és megbízható lett.

4. Deployment: A rendszer telepítése az Azure felhőszolgáltatásra történt, ahol az API az `api.bus4u.online` domainen érhető el. A PostgreSQL adatbázis és a virtuális gép (VM) külön szolgáltatásként működik, biztosítva a skálázhatóságot és a megbízhatóságot.

5. Jövőbeli fejlesztések: A jövőbeli fejlesztések között szerepel a rendszer funkcionálisának bővítése, például új szolgáltatások bevezetése, a felhasználói élmény javítása érdekében. További cél a rendszer biztonságának növelése, például a hitelesítési és jogosultsági mechanizmusok fejlesztésével.

Összességében a Bus4U API megvalósítása sikeres volt, és a dolgozat során végrehajtott lépések hozzájárultak a hatékony és megbízható busz menedzselő rendszer kialakí-

tásához. A projekt során szerzett tapasztalatok és tanulságok értékes alapot jelentenek a jövőbeli fejlesztésekhez és a hasonló rendszerek tervezéséhez.

# Irodalomjegyzék



A függelék

Függelék

függelék